



Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la carrera:

Ingeniería en Informática.

Nombre de la asignatura:

Programación cliente - servidor.

Semestre y grupo:

7AI

Actividad:

Registro de práctica #2 Matriz de Decisión Arquitectónica.

Nombre del alumno:

Aaron Jesús Eloy Aguilar León.

Chi Chablé Miguel Eduardo.

Ek Noh Edgar Ernesto.

Nombre del docente:

Jenner Noe Che Méndez.

Lugar y fecha:

San Francisco de Campeche, Campeche a 03 de Octubre de 2026.

Objetivo.

Construir y defender una matriz de decisión arquitectónica ponderada para comparar mecanismos de comunicación cliente-servidor (REST, gRPC, WebSocket, SSE y Eventos), seleccionando y justificando la alternativa técnica o combinación tecnológica más adecuada para un escenario específico con base en requisitos, restricciones y criterios explícitos.

1.- Descripción del caso.

Desarrollo de un módulo de comunicación instantánea integrado a la plataforma institucional del Instituto Tecnológico de Chiná. El sistema debe permitir a estudiantes y docentes intercambiar mensajes en tiempo real, visualizar indicadores de presencia en línea (online/offline) y mostrar eventos dinámicos como "escribiendo..." en salas de chat grupales por asignatura o mensajes individuales.

2.- Requisitos funcionales.

- Enviar y recibir mensajes instantáneos en conversaciones individuales y grupales.
- Notificar en tiempo real cuándo un usuario está conectado/desconectado.
- Mostrar el indicador dinámico de "escribiendo..."
- Consultar el historial de conversaciones pasadas.
- Gestionar la autenticación de usuarios e inicio de sesión.

3.- Requisitos No Funcionales.

- Baja latencia: Entrega de mensajes en menos de 200 ms.
- Comunicación Bidireccional: Habilidad de enviar y recibir datos simultáneamente sobre una misma conexión persistentemente abierta.
- Escalabilidad: Soporte de conexiones concurrentes para la comunidad estudiantil.
- Mantenibilidad: Facilidad de evolución del código sin afectar la infraestructura existente.

4.- Tecnologías evaluadas.

- REST (Representational State Transfer).
- gRPC (gRPC Remote Procedure Calls).
- WebSocket.
- SSE (Server-Sent Events).
- Eventos y mensajería (Broker / Pub-Sub).

5.- Explicación breves de las tecnologías.

REST: Patrón petición-respuesta síncrono sobre HTTP/HTTPS, ideal para operaciones CRUD.

gRPC: Protocolo de llamadas remotas de alto rendimiento mediante HTTP/2 y contratos binarios (Protocol Buffers).

WebSocket: Protocolo con conexión persistente y bidireccional full-duplex sobre TCP.

SSE: Flujo de datos continuo y unidireccional transmitido únicamente desde el servidor hacia el cliente sobre HTTP.

Eventos: Arquitectura asíncrona basada en publicación/suscripción (Pub/Sub) mediante un intermediario (broker).

6.- Criterios y justificación.

C1.- Tiempo real (20%): Esencial para la inmediatez en el envío de mensajes.

C2.- Bidireccionalidad (20%): Requisito crítico para recibir mensajes e indicadores simultáneamente.

C3.- Compatibilidad web/móvil (10%): Necesario para operar en navegadores y apps móviles.

C4.- Rendimiento (10%): Minimizar la sobrecarga de encabezados HTTP en cada mensaje.

C5.- Escalabilidad (10%): Sostener múltiples conexiones socket concurrentes.

C6.- Facilidad de implementación (5%): Evalúa la simplicidad para el equipo de desarrollo.

C7.- Bajo acoplamiento (5%): Independencia entre capas.

C8.- Mantenimiento (5%): Facilidad de soporte a largo plazo.

C9.- Ecosistema (5%): Disponibilidad de bibliotecas (ej. Socket.io).C10. Adecuación al caso (10%): Grado en que la tecnología responde de forma directa a la necesidad de chat.

7.- Ponderaciones.

Criterio	Peso (%)	Peso Decimal
Tiempo real	20%	0.20
Bidireccionalidad	20%	0.20
Compatibilidad	10%	0.10
Rendimiento	10%	0.10
Escalabilidad	10%	0.10
Adecuación al caso	10%	0.10
Facilidad	05%	0.05
Bajo acoplamiento	05%	0.05
Mantenimiento	05%	0.05
Ecosistema	05%	0.05
Total	100%	1.00

8.- Matriz de evaluación (valor de 1 – 5).

Criterio	Peso	REST	gRPC	Websocket	SSE	Eventos
Tiempo real	20%	2	4	5	4	5
Bidireccionalidad	20%	1	4	5	1	3
Compatibilidad	10%	5	3	5	4	2
Rendimiento	10%	3	5	4	4	5

Escalabilidad	10%	4	5	4	4	5
Adecuación al caso	10%	2	3	5	2	3
Facilidad	05%	5	3	4	4	2
Bajo acoplamiento	05%	4	4	3	3	5
Mantenimiento	05%	5	3	4	4	2
Ecosistema	05%	5	4	5	4	5

9.- Calculo Ponderado (Valor por peso).

Formula que se utilizó: Puntuación = Valor * Peso Decimal.

Criterio	Peso	REST	gRPC	Websocket	SSE	Eventos
Tiempo real	0.20	0.40	0.80	1.00	0.80	1.00
Bidireccionalidad	0.20	0.20	0.80	1.00	0.20	0.60
Compatibilidad	0.10	0.50	0.30	0.50	0.40	0.20
Rendimiento	0.10	0.30	0.50	0.40	0.40	0.50
Escalabilidad	0.10	0.40	0.50	0.40	0.40	0.50
Adecuación al caso	0.10	0.20	0.30	0.50	0.20	0.30
Facilidad	0.05	0.25	0.15	0.20	0.20	0.10
Bajo acoplamiento	0.05	0.20	0.20	0.15	0.15	0.25
Mantenimiento	0.05	0.25	0.15	0.20	0.20	0.10
Ecosistema	0.05	0.25	0.20	0.25	0.20	0.25
Total	1.00	2.95	3.90	4.60	3.15	3.80

10.- Rankings de tecnologías.

- WebSocket: 4.60 / 5.00 (Primer lugar)

- gRPC: 3.90 / 5.00 (Segundo lugar)
- Eventos: 3.80 / 5.00
- SSE: 3.15 / 5.00
- REST: 2.95 / 5.00

11.- Tecnología a seleccionar.

La tecnología que se seleccionó fue la de WebSocket.

12.- Justificación Arquitectónica.

WebSocket obtiene la mayor puntuación (4.60) debido a que los criterios determinantes del escenario (Tiempo real y Bidireccionalidad) representan el 40% del peso total de la evaluación.

Al establecer un canal de comunicación persistente sobre TCP, permite que tanto el cliente como el servidor se envíen paquetes con una sobrecarga de encabezados mínima y sin la necesidad de realizar peticiones de sondeo (polling), es entonces que, gracias a esto, se garantiza una experiencia fluida para las notificaciones de estado ("escribiendo...", usuarios conectados) y la recepción instantánea de mensajes.

13.- Alternativa secundaria.

Podemos tener en cuenta la opción que quedó en segundo lugar, que es el gRPC (puntuación 3.90).

Fue descartada como tecnología principal debido a su menor compatibilidad nativa directa en el navegador web cliente y la complejidad adicional para gestionar la capa de gRPC-Web.

14.- Posible combinación tecnológica.

En una arquitectura de producción real no se utiliza exclusivamente WebSocket para todo el módulo.

La combinación óptima y más adecuada es REST + WebSocket debido a funciones en particular de cada uno, tales son:

REST: Se emplea para autenticación de usuarios, carga de archivos adjuntos (imágenes/documentos) y consulta del historial de chats pasados.

WebSocket: Se activa únicamente para sostener el canal interactivo en tiempo real durante la sesión activa de chat.

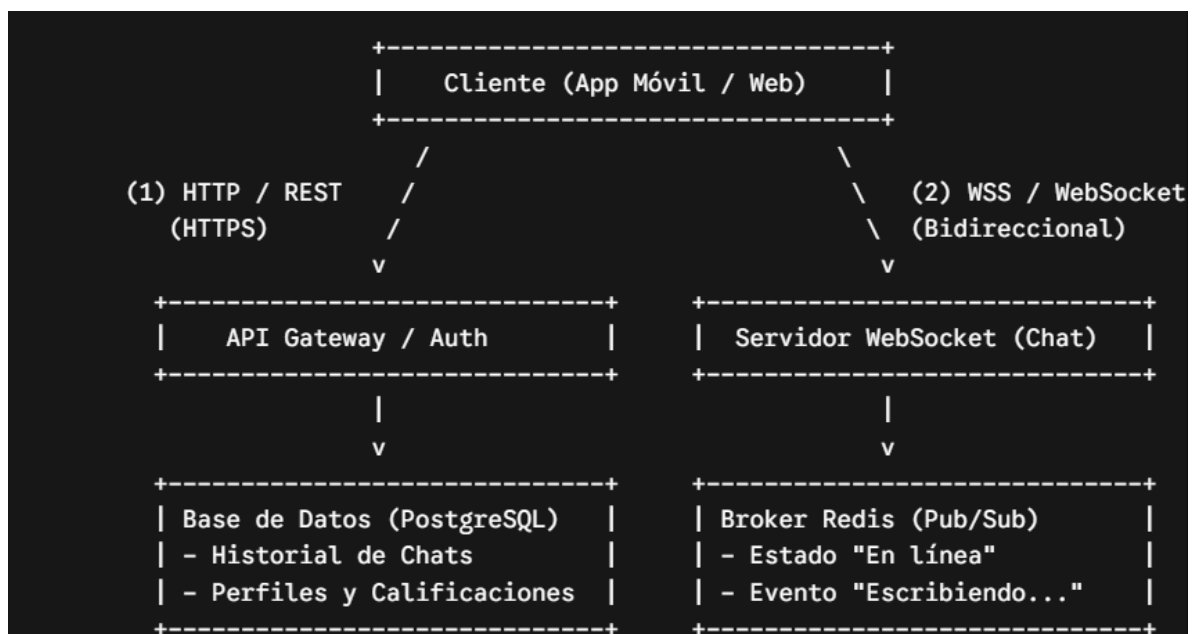
15.- Conclusión.

La selección tecnológica no debe responder a modas, sino al análisis explícito de las restricciones del problema.

Para un sistema de chat, el modelo tradicional de petición-respuesta (REST) resulta insuficiente debido a la sobrecarga y latencia que genera el sondeo recurrente.

La matriz ponderada demuestra matemáticamente que WebSocket es el protocolo mejor capacitado para este dominio funcional, equilibrando el rendimiento bidireccional con una amplia compatibilidad dentro del ecosistema web y móvil; así mismo para la finalidad con la que se establece.

Se anexa diagrama sencillo de arquitectura:



Cuestionario para responder:

1.- ¿Existe una tecnología que sea mejor en todos los escenarios?

No, ya que cada tecnología resuelve problemas distintos, por ejemplo REST destaca en operaciones CRUD simples, WebSocket en comunicación bidireccional en tiempo real, y gRPC en comunicación interna entre microservicios de alto rendimiento.

La "mejor" opción siempre depende de las restricciones y requisitos específicos del caso.

2.- ¿Por qué REST sigue siendo tan utilizado?

Por su simplicidad, estandarización sobre HTTP, compatibilidad universal con cualquier cliente o navegador web, soporte para almacenamiento en caché (caching) y un ecosistema maduro para operaciones sin estado (stateless).

En el caso de chat académico se usa para autenticación y gestión del historial.

3.- ¿Cuándo WebSocket sería innecesario?

Cuando la aplicación solo requiere consultar información de forma esporádica (operaciones de lectura/escritura estándar como ver materias o calificaciones) o cuando las actualizaciones ocurren raramente y basta con notificaciones unidireccionales desde el servidor.

Mantener conexiones TCP abiertas en esos casos consume recursos innecesariamente, por esa razón no sería apto.

4.- ¿Qué diferencia fundamental existe entre WebSocket y SSE?

WebSocket: Ofrece comunicación bidireccional (full-duplex); cliente y servidor pueden enviar y recibir mensajes simultáneamente.

SSE (Server-Sent Events): Es unidireccional; solo el servidor puede enviar un flujo de datos continuo hacia el cliente sobre HTTP.

5.- ¿Por qué gRPC es atractivo para microservicios?

Por su alto rendimiento basado en HTTP/2, la serialización binaria ultrarrápida mediante Protocol Buffers y el tipado estricto con contratos explícitos, lo que reduce la latencia de comunicación interna entre servicios backend.

6.- ¿Qué significa que una arquitectura esté desacoplada?

Significa que sus componentes o módulos son independientes entre sí y se comunican mediante interfaces o eventos bien definidos.

Un cambio interno o caída en un servicio (ej. la búsqueda de historial) no detiene ni afecta el funcionamiento de otros servicios (ej. el canal de chat en vivo).

7.- ¿Por qué los eventos favorecen el desacoplamiento?

Porque el emisor del evento no necesita conocer quiénes son los receptores ni cuántos hay, solo publica el mensaje en un canal/topic y los servicios interesados (suscriptores) lo procesan de forma asíncrona.

8.- ¿Qué problemas introduce una arquitectura orientada a eventos?

Mayor complejidad operacional, dificultad para depurar y rastrear flujos de mensajes (tracing), posibilidad de mensajes duplicados o desordenados, y eventual inconsistencia de datos si no se gestionan adecuadamente las transacciones.

9.- ¿Por qué los pesos de una matriz pueden cambiar el resultado?

Porque los pesos reflejan la prioridad del negocio o del caso de uso.

Ya que al cambiar la importancia relativa de los criterios (por ejemplo, darle 40% a Seguridad en lugar de Tiempo real), la ponderación matemática aparte de alterarse, se modificará qué tecnología obtiene la puntuación más alta.

10.- ¿Quién debe definir esos pesos?

El equipo de arquitectura de software en conjunto con los líderes de producto y expertos del dominio del negocio, asegurando que las prioridades técnicas se alineen con los objetivos y restricciones del proyecto.

11.- ¿Qué ocurre si dos tecnologías obtienen prácticamente la misma puntuación?

Se debe realizar un análisis cualitativo secundario evaluando criterios complementarios como la curva de aprendizaje del equipo, los costos de infraestructura o la madurez de las bibliotecas disponibles para tomar una decisión final.

12.- ¿Podrían utilizarse REST y WebSocket en el mismo sistema?

Sí, y es la práctica recomendada, en el caso que escogí que es el chat académico, REST gestiona autenticación, perfiles y la consulta inicial del historial, mientras WebSocket sostiene la sesión activa de chat para intercambio de mensajes instantáneos y estados en tiempo real.

13.- ¿Podrían utilizarse REST y eventos simultáneamente?

Sí, el cliente invoca servicios mediante peticiones HTTP REST, y el servidor internamente publica un evento en un broker (como Redis o RabbitMQ) para procesar tareas asíncronas en segundo plano, como enviar notificaciones push a los dispositivos.

14.- ¿Qué criterios serían importantes para una aplicación móvil?

- Consumo de batería y procesamiento: Uso eficiente de recursos.
- Eficiencia en el consumo de datos móviles: Minimización de payloads y sobrecarga de encabezados.
- Manejo de conectividad intermitente: Capacidad de reconexión automática y sincronización offline.

15.- ¿Qué criterios cambiarían para una aplicación bancaria?

Aumentaría radicalmente el peso de la Seguridad y Cifrado, Integridad de Transacciones (cumplimiento ACID), Auditoría y Trazabilidad de operaciones, y la Disponibilidad/Tolerancia a fallos, relegando a un segundo término criterios como la facilidad de desarrollo o ecosistema.